Homework 3: Inverse Kinematics

Fall 2026 | Due 10/16/2026 at 11:59 PM

Assignment at a glance: Derive a geometric, partial inverse-kinematics solution for a top-down grasp with the SO-101, implement it in Python, and validate reachable targets by composing IK with your Homework 2 forward kinematics.

1. Objectives

- Decompose a spatial inverse-kinematics problem into base rotation and planar geometry.
- Compute a wrist-center target for a prescribed top-down tool orientation.
- Derive one consistent elbow-up solution for shoulder, elbow, and wrist flex.
- Detect unreachable or numerically invalid targets before evaluating trigonometric expressions.
- Implement inverse kinematics in Python and validate it using forward kinematics.

2. Files Provided

File	Purpose
student_ik.py	The only new Python file you edit and submit.
provided_fk.py	Provides the reference Homework 2 forward-kinematics implementation used by the test scripts. You may use your student_fk.py if you prefer.
test_single_ik.py	Tests one target position and reports FK-of-IK position and orientation error.
test_ik.py	Evaluates your IK over a collection of reachable targets.
mujoco_oracle.py	Provides the MuJoCo reference pose used by the test scripts. Do not edit.
robot-assets/scene.xml	Loads the SO-101 model used by the oracle.

3. Robot Model and Given Dimensions

Use the same SO-101 robot, base frame, tool frame, joint names, joint signs, and dimensions from Homework 2. The figures below show the zero configuration and the idealized tool frame used in the derivation.

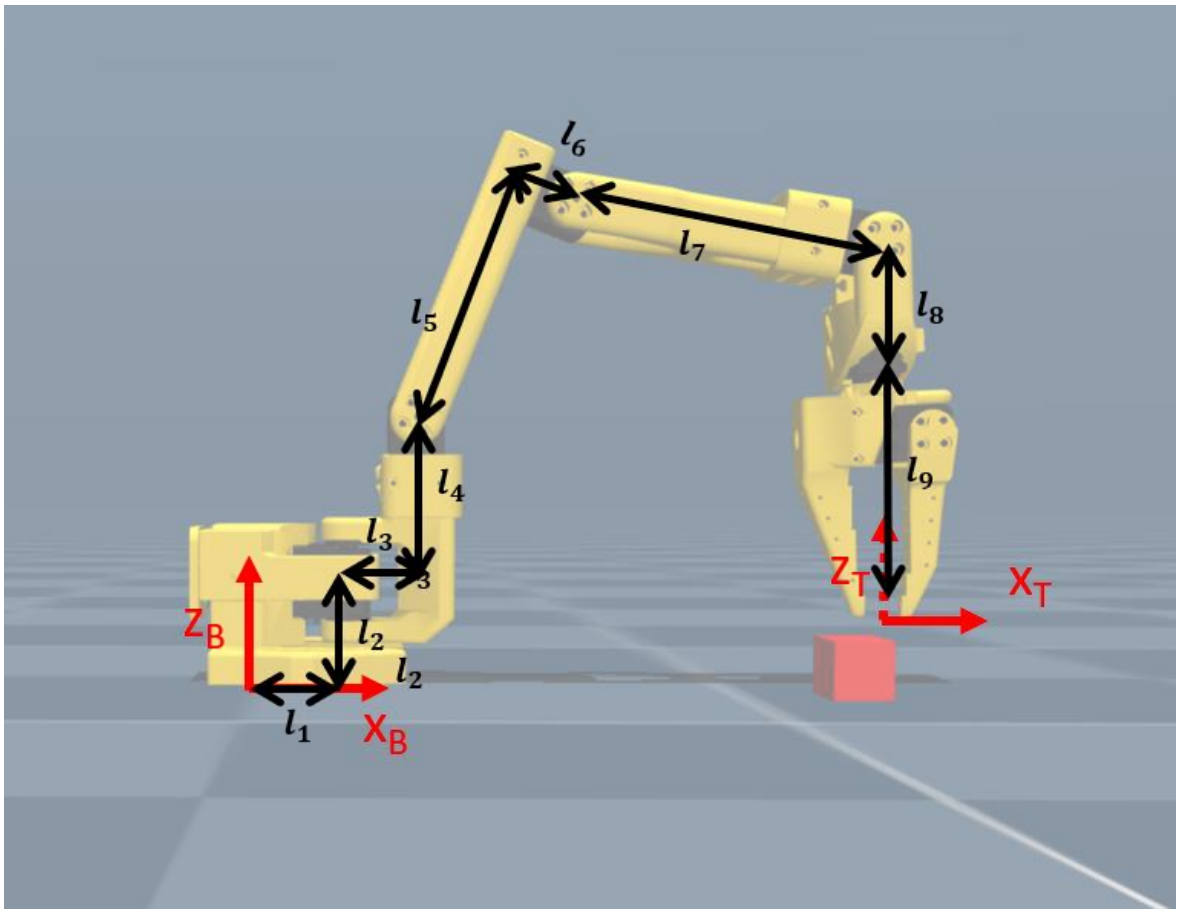


Figure 1. Annotated side views of the SO-101 used in Homework 3. The gripper is oriented down.

Parameter	Value (m)	Description
l1	0.0388	Base horizontal offset
l2	0.0624	Base vertical offset
l3	0.0304	Shoulder horizontal offset
l4	0.0542	Shoulder height
l5	0.1126	Upper-arm long dimension
l6	0.0280	Upper-arm short offset
l7	0.1349	Forearm length
l8	0.0611	Wrist length
l9	0.1004	Gripper length

4. Assignment Model and Required Pose

The input is a desired tool-tip position expressed in the given base frame:

target = [X, Y, Z]^T

- The gripper must point straight down for a top-down grasp.
- Follow one consistent elbow-up solution. You are not required to return every possible IK solution.
- Set wrist roll to a fixed value of 0 rad unless the starter code states otherwise.
- All distances are in meters and all angles are in radians.
- The requested configuration must respect the joint limits used by the supplied simulator and tests.

Important note: Your equations and code must use the same base frame, tool axis, joint signs, and fixed offsets as your Homework 2 FK. A geometrically correct solution in a different convention will not pass the tests.

For the Homework 2 tool-frame convention, the physical gripper points along the negative tool z-axis. Therefore, a top-down grasp requires the tool z-axis, expressed in the base frame, to equal $[0,0,1]^T$. Equivalently, the negative tool z-axis must point along the negative base z-axis.

5. Task 1: Derive the Partial Inverse Kinematics (35 points)

Submit a handwritten-and-scanned or electronically written derivation. Show enough intermediate geometry that another person can follow your sign choices. The following steps will help you:

1. Base rotation θ_1: Draw the arm from above in the base XY plane and derive θ_1 (shoulder_pan) using atan2. Account for the fixed horizontal offset between the given base origin and the pan axis. It will probably help to have $\theta_1 \neq 0$.

2. Wrist-center position: Draw the arm from the side. It will help to draw it in an elbow-up configuration (i.e. joint angles are not 0). Make sure the gripper is pointed straight down. Move up from the desired tool tip coordinates to find the wrist center. Then express the wrist center coordinates relative to the *shoulder*. You will probably need to refer to the top-down for x_w, y_w and side drawings for z_w.

3. Effective planar links: Replace the bent upper-arm geometry by a single effective length l_a and a fixed angular offset β. Define both quantities from l5 and l6. (e.g. replace l_5 and l_6 by a single "diagonal" length l_a).

4. Elbow angle θ_3: Redraw from the shoulder to the wrist center, with the frame F_s in the shoulder. You should have a triangle of l_a, l_7 and a distance P from the shoulder to the wrist. Solve for P. Also solve for the constant angle β between l_6 and l_a.

Use the law of cosines to derive the angle α between l_a and l_7. For the next step, I found it helpful to draw both with $\theta_3 = 0$ and $\theta_3 < 0$. If $\theta_3 = 0$, then you should see that $\alpha = \pi - \beta$. If $\theta_3 < 0$, you should see that $\alpha = \pi - \beta - \theta_3$. Be careful about the sign of θ_3! Solve for the elbow_flex term θ_3. (I gave you all the equations you need, but sketch it out so it makes sense to you).

5. Shoulder angle θ_2: Use the wrist-center elevation and the shoulder-elbow-wrist triangle (l_a, l_7, P) to derive θ_2. Note that $\theta_2 = \pi - \gamma - \delta - (\pi - \beta)$, where γ is the angle between l_a and P and δ is the angle from the horizontal to the wrist center.

6. Wrist angle θ_4: Derive wrist_flex θ_4 so the required tool axis points along the base z-axis. I.e. the gripper faces down but the z_T axis points up. Your expression must agree with your Homework 2 transform chain. This is actually quite simple, since Z_T and Z_B are already aligned if the gripper faces down. This might be why the designers chose Z_T oriented the way they did. Just solve for θ_4 to compensate for θ_2 and θ_3.

7. Final result: Give equations for all five joint angles as a function of the target points, robot link lengths, etc.

6. Task 2: Implement inverse_kinematics(target) (15 points)

Open student_ik.py and implement the required function without changing its name or input/output format.

```
q, reachable = inverse_kinematics(target)
target.shape == (3,)
```

q is a dictionary containing the five Homework 2 joint names.
reachable is a Boolean.

- Return a finite result only when a valid elbow-up solution exists within the required model and joint limits.
- Clip cosine-law values only to protect against small floating-point roundoff near ±1. Do not use clipping to force a substantially unreachable target to appear reachable.
- Do not hard-code joint angles for the supplied examples.
- You may call your Homework 2 dh_transform or forward_kinematics functions, but inverse_kinematics itself must implement the geometric solution.

7. Validation

7.1 Test one target

1. Edit TARGET_POSITION in test_single_ik.py.
2. Run the script from the lerobot-mujoco directory using the command supplied with the starter files.
3. Inspect the returned joint angles, reachability flag, reconstructed tool position, and downward-orientation error.
4. Repeat with several interior-workspace targets and at least one unreachable target.

7.2 Required round-trip check

For a reachable target p_{target}, you're the test will evaluate

$$q = IK(p_{target}), \quad (p_{check}, R_{check}) = FK(q)$$

and compare p_{check} with p_{target}. The orientation check will verify the third column of R_{check} is ≈ [0 0 1]^T.

Expected nonzero error: As in Homework 2, the MuJoCo oracle measures a physical feature on the gripper while the derivation uses an idealized tool frame. Small, consistent discrepancies can remain even when the equations and code are internally consistent.

8. Graduate Requirement / Undergraduate Extra Credit (15 points)

Required for CSE 5364 and extra credit for CSE 4360: derive the linear-velocity Jacobian that maps the first four joint rates to the tool-tip velocity expressed in the base frame. You may obtain it by differentiating your Homework 2 position function or by using joint axes and origins from the transform chain. Clearly define the matrix dimensions and frame of expression.

9. Submission Requirements

Submit two files

- <last_name>_<first_name>_HW3.pdf that includes your solutions equations for all five joint angles, along with any drawings, figures, notes, etc as will help the graders understand your work.
- student_ik.py – the TA will run test_ik.py with your code.

AI Use Disclaimer: I used Microsoft Copilot (GPT-5) to help test, clarify, and add comments to the provided code and improve the clarity and organization of this document.